

Advanced Machine Learning

DATA 642

Lecture 5 — SVMs Under the Hood

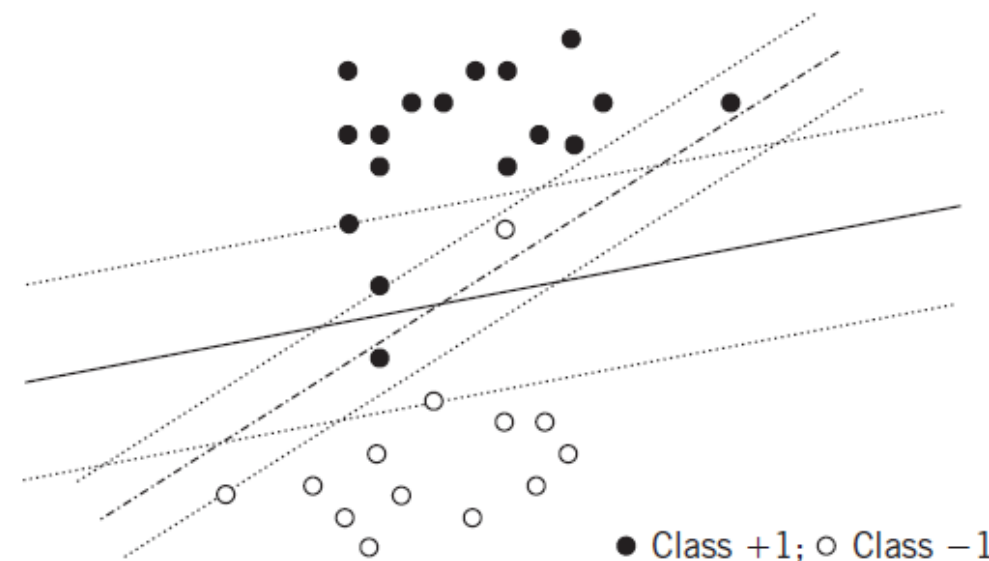
*Figures and notes are from the book Mathematics for Machine Learning by A. Aldo Faisal, Cheng Soon Ong, and Marc Peter Deisenroth, Machine Learning A Bayesian and Optimization Perspective, Sergios Theodoridis, Elsevier, and Hands-On Machine Learning with Scikit-Learn Keras and TensorFlow, Aurelien Geron, O'Reilly

The main idea

Imagine that the open and filled circles in the following figure are houses in two nearby villages and that a road must be constructed in between the two villages. One has to decide where to construct the road so that it will be as wide as possible and incur the least cost (in the sense of demolishing the smallest number of houses). No sensible engineer would choose the "dash-dotted" option.

The idea is similar with designing a classifier. It should be "placed" between the highly populated (high probability density) areas of the two classes and in a region that is sparse in data, leaving the largest possible margin.

This is dictated by the requirement for good generalization performance that any classifier has to exhibit. That is, the classifier must exhibit good error performance when it is faced with data outside the training set.



Linearly separable classes

Assuming linearly separable classes, there is an infinity of linear classifiers that solve the classification task exactly, without committing errors on the training set (see Fig. 11.14). It is easy to see, and it will become apparent very soon, that from this infinity of hyperplanes that solve the task, we can always identify a subset such as

$$y_n(\boldsymbol{\theta}^\top \mathbf{x}_n + \theta_0) \geq 1, \quad n = 1, \dots, N \quad (1)$$

Thus, for linear separable classes we form the following optimization task

$$\begin{aligned} & \underset{\boldsymbol{\theta} \in \mathbb{R}^l}{\text{minimize}} && \frac{\|\boldsymbol{\theta}\|^2}{2} \\ & \text{subject to} && y_n(\boldsymbol{\theta}^\top \mathbf{x}_n + \theta_0) \geq 1, \quad n = 1, \dots, N \end{aligned} \quad (2)$$

In other words, from this infinity of linear classifiers, which can solve the task and classify correctly all training patterns, our optimization task selects the one that has minimum norm. The norm $\|\boldsymbol{\theta}\|$ is directly related to the margin formed by the respective classifier.

The solution

The constrained optimization problem, known as the **primal problem**, has a counterpart called the **dual problem**. While the dual solution generally provides a lower bound to the primal solution, in certain conditions, such as convexity of the objective function and differentiability and convexity of inequality constraints, the solutions can coincide. Fortunately, the SVM problem satisfies these conditions, offering the flexibility to solve either the primal or the dual problem.

To obtain a solution we define the Lagrangian function

$$\mathcal{L}(\boldsymbol{\theta}, \theta_0, \lambda) = \frac{1}{2} \|\boldsymbol{\theta}\|^2 - \sum_{n=1}^N \lambda_n (y_n (\boldsymbol{\theta}^\top \mathbf{x}_n + \theta_0) - 1), \quad (3)$$

Using the KKT (Karush-Kuhn-Tucker) conditions we have

$$\nabla_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}, \theta_0, \lambda) = \mathbf{0} \Rightarrow \hat{\boldsymbol{\theta}} = \sum_{n=1}^N \lambda_n y_n \mathbf{x}_n (*) \quad (4)$$

and

$$\nabla_{\theta_0} \mathcal{L}(\boldsymbol{\theta}, \theta_0, \lambda) = 0 \Rightarrow \sum_{n=1}^N \lambda_n y_n = 0, \quad \lambda_n (y_n (\boldsymbol{\theta}^\top \mathbf{x}_n + \theta_0) - 1) = 0, \quad \lambda_n \geq 0, \quad n = 1, \dots, N. \quad (5)$$

The Lagrange multipliers are obtained via the dual representation form after plugging (*) into $\mathcal{L}(\boldsymbol{\theta}, \theta_0, \lambda)$ using a quadratic programming (QP) solver. The dual representation form is given by

$$\begin{aligned} & \underset{\lambda}{\text{minimize}} && \frac{1}{2} \sum_{n=1}^N \sum_{m=1}^N \lambda_n \lambda_m y_n y_m \mathbf{x}_n^T \mathbf{x}_m - \sum_{n=1}^N \lambda_n \\ & \text{subject to} && \lambda_n \geq 0, \text{ and } \sum_{n=1}^N \lambda_n y_n = 0, \quad n = 1, \dots, N. \end{aligned} \tag{6}$$

Once we have a solution for $\hat{\boldsymbol{\lambda}}$ (using a QP solver), the dual solution to the primal solution is given by

$$\hat{\boldsymbol{\theta}} = \sum_{n=1}^{N_s} \lambda_n y_n \mathbf{x}_n, \tag{7}$$

where N_s is the number of non-zero Lagrange multipliers. Only the only the Lagrange multipliers associated with the nearest-to-the-classifier points, that is, those points satisfying the constraints with equality ($y_n(\boldsymbol{\theta}^\top \mathbf{x}_n + \theta_0) = 1$), are nonzero. These are known as the support vectors. The estimate of the bias term, $\hat{\theta}_0$, is obtained by selecting all constraints with $\lambda_n \neq 0$, corresponding to $(y_n(\hat{\boldsymbol{\theta}}^\top \mathbf{x}_n + \hat{\theta}_0) = 1)$.

Notes

- The solution we obtain for SVM is unique. The cost function is strictly convex!
- The dual problem is faster to solve than the primal one!
- The dual problem makes the kernel trick possible! This then implies that for more general RKHS case we have the following

$$\hat{y}(\mathbf{x}) = \sum_{n=1}^{N_s} \lambda_n y_n \kappa(\mathbf{x}, \mathbf{x}_n) + \theta_0 \quad (8)$$

Non-separable classes

When the classes are overlapping we compute a hyperplane so as to

$$\begin{aligned} &\underset{\boldsymbol{\theta} \in \mathbb{R}^l}{\text{minimize}} && \frac{\|\boldsymbol{\theta}\|^2}{2} + C \sum_{n=1}^N \xi_n \\ &\text{subject to} && y_n(\boldsymbol{\theta}^\top \mathbf{x}_n + \theta_0) \geq 1 - \xi_n; \quad \xi_n \geq 0 \end{aligned}$$

A margin error is committed if $y_n(\boldsymbol{\theta}^\top \mathbf{x}_n + \theta_0) < 1$ corresponding to $\xi_n > 0$. If $\xi_n = 0$ then $y_n(\boldsymbol{\theta}^\top \mathbf{x}_n + \theta_0) \geq 1$ and there is no contribution in the cost function. Thus, the goal of the optimization task is to drive as many ξ_n 's to zero as possible.

Note

Optimization is achieved in a trade-off rationale; the user-defined parameter C controls the influence of each of the two contributions to the minimization task. If C is large, the resulting margin will be small, in order to commit a smaller number of margin errors. If C is small, the opposite is true.

Choice of hyper-parameters

One of the main issues associated with SVM/SVRs is the choice of the parameter C , which controls the relative influence of the loss and the regularizing parameter in the cost function. Although some efforts have been made in developing theoretical tools for the respective optimization, the path that has survived in practice is that of cross-valuation techniques against a test data set.

The other main issue is the choice of the kernel function. Different kernels lead to different performance. If for instance the Gaussian kernel is employed, one must have access to a large number of training data, in order to fill in the input space densely enough, so as to be able to obtain a good enough approximation of such a function. This brings into the scene another critical factor in machine learning, related to the size of the training set. This is because a kernel $\kappa(\mathbf{x}, \mathbf{x}_n)$ matches $\mathbf{x}$ to the training sample point $\mathbf{x}_n$.

1. **Linear Kernel** ($\kappa(\mathbf{x}, \mathbf{x}') = \mathbf{x}^\top \mathbf{x}'$):

- **Usage:** Use when the data is linearly separable or when there is a large number of features relative to the number of samples.
- **Example:** Classification tasks where the decision boundary is expected to be a straight line in the input space.

2. **Polynomial Kernel** ($\kappa(\mathbf{x}, \mathbf{x}') = (\mathbf{x}^\top \mathbf{x}' + c)^d$):

- **Usage:** Suitable for problems where the decision boundary is expected to be polynomial.
- **Example:** Image classification where the boundaries between classes might have a polynomial shape.

3. **Gaussian RBF Kernel** ($\kappa(\mathbf{x}, \mathbf{x}') = \exp\left(-\frac{\|\mathbf{x}-\mathbf{x}'\|^2}{2\sigma^2}\right)$):

- **Usage:** Effective for capturing complex nonlinear relationships between features.
- **Example:** Financial data analysis where relationships between variables might be nonlinear and difficult to model with linear methods.

4. **Sigmoid Kernel** ($\kappa(\mathbf{x}, \mathbf{x}') = \tanh(\alpha \mathbf{x}^\top \mathbf{x}' + c)$):

- **Usage:** Useful for problems where the data exhibits periodic patterns or where the relationship between features is expected to be similar to a sigmoid function.
- **Example:** Time series analysis, where the data might exhibit cyclic behavior.